

- DROPKAST — Production Implementation Plan
 - Phase 0 — Foundation (3–5 days)
 - Phase 1 — Core Distro (1–2 weeks)
 - Phase 2 — AI as the Differentiator (1 week)
 - Phase 3 — Money Flow (1 week)
 - Phase 4 — Creator Economy (1 week)
 - Phase 5 — Polish & Type Safety (3–5 days)
 - Phase 6 — Observability (1–2 days)
 - Phase 7 — Security Hardening
 - Out of scope (don't build until validated)
 - Suggested timeline
 - Definition of "shippable v1"

DROPKAST — Production Implementation Plan

Ordered by **what unlocks revenue or competitive advantage**, not by "what every web app theoretically needs."

The current state: a Vite + Express demo with mock auth (`localStorage`), an in-memory database that wipes on every Vercel cold start, fake DSP delivery (`setTimeout`), and a chat assistant that returns hardcoded strings. None of that is shippable to paying artists. Everything below is what closes that gap.

Phase 0 — Foundation (3–5 days)

You can't sell anything until artists can actually log in and their data survives a deploy. Do this first or nothing else matters.

- ☒ **Replace mock auth with Supabase Auth** — wired with graceful fallback to legacy `localStorage` when keys missing
- ☒ **Replace in-memory db** — `api/_store.ts` abstraction uses Drizzle/Postgres when `DATABASE_URL` is set, in-memory bucket otherwise
- ☒ **Add Drizzle ORM** — schema in `db/schema.ts` (13 tables), client in `db/client.ts`

- ☐ **Row-Level Security policies** — needs Supabase project to write
- ☐ **Move file uploads from Cloudinary to Supabase Storage** — optional refactor; Cloudinary works
- ☒ **Server-side env vars audit** — `GEMINI_API_KEY` leak removed from `vite.config.ts`
- ☒ **Zod schemas** — `api/_schemas.ts` covers all endpoints, validated via `validate()` middleware

Why this first: Supabase gives you auth + Postgres + storage + RLS in one signup. Replaces 14 items from the old plan in an afternoon.

Phase 1 — Core Distro (1–2 weeks)

This is the part of the app that has to actually work for DropKast to be "a music distro" instead of "a UI mockup."

- [~] **Real DSP delivery via RouteNote API** — pluggable adapter scaffolded in `api/_dsp-delivery.ts` (default = simulator). Real RouteNote impl pending partner contract + key.
- ☒ **ISRC code generation** — `api/_codes.ts` generates valid `CC-XXX-YY-NNNNN` format. Override registrant via `ISRC_REGISTRANT` env var when RIAA block purchased.
- ☒ **UPC code per release** — same module, valid mod-10 check digit. Override prefix via `UPC_COMPANY_PREFIX` when GS1 block purchased.
- ☒ **Audio file validation on upload** — `api/_audio-validate.ts` checks duration, sample rate, bit depth, container, MP3 bitrate
- ☒ **Loudness normalization check** — peak (dBFS) + integrated LUFS read from ReplayGain/R128 metadata; flags over-mastered + too-quiet tracks. Full ITU BS.1770 wasm analysis is a future enhancement.
- ☒ **Cover art validation** — $\geq 3000 \times 3000$, square, JPEG/PNG/WebP. Native dimension reading without heavy image lib.
- ☒ **Release lifecycle state machine** — `api/_release-lifecycle.ts` enforces `draft` $\rightarrow$ `submitted` $\rightarrow$ `in_review` $\rightarrow$ `approved` $\rightarrow$ `delivering` $\rightarrow$ `live` (+ `rejected`, `taken_down`)
- ☒ **Takedown / metadata edit flow** — `POST /api/releases/:id/takedown` and `PATCH /api/releases/:id/metadata` both wired, audit-logged, route through DSP adapter for DDEX update.

- **☑Release scheduling** — future `releaseDate` parks the release in `approved` state until that date

Why this matters: DistroKid charges \$20/year for exactly this. It's the table-stakes feature. Without it DropKast is a campaign tool, not a distro.

Phase 2 — AI as the Differentiator (1 week)

The thing that makes DropKast different from DistroKid/TuneCore. Build the AI layer once the foundation is real (so the assistant can read real data via tool use).

- **☑Swap Gemini → Claude (Anthropic) in `src/services/aiService.ts`** — done, Sonnet for reasoning + Haiku for cheap stuff
- **☑Streaming chat endpoint** — `POST /api/ai/chat` (SSE) with 8 swappable brain backends (Anthropic / Moonshot / OpenAI / Google / NVIDIA / Groq / Cerebras / OpenRouter)
- **☑Tool definitions for the assistant** — `get_my_releases`, `get_release_analytics`, `get_active_campaigns`, `get_influencers`, `get_anr_submissions`
- **☑Replace `setTimeout` mock in `AIAssistant.tsx`** — real SSE stream rendering with tool-call previews
- **☑Prompt caching** — `cache_control: ephemeral` on the system prompt
- **☑Real A&R critique on `/anr` page** — 1-10 score + 6-section markdown critique authored by `ar-critic` persona
- **☑Real campaign strategy generation** — Claude call via `campaign-director` persona with 60/30/10 budget rule baked in
- **☑Cost guardrails** — `api/_ai-budget.ts` enforces \$1/day default per user (override via `AI_DAILY_BUDGET_CENTS`); 429 with friendly message
- **☑Music-pro personas (*scope addition*)** — 11 system prompts authored from real industry expertise, in `personas/` + `src/lib/ai-personas.ts`

Why this works: Tool use + streaming = the assistant can answer "how's Buddy Kay doing on TikTok this week?" with real numbers. Without tool use it's just another chatbot.

Phase 3 — Money Flow (1 week)

If artists upload music and the splits never pay, you have a lawsuit, not a product.

- [~] **Stripe Connect onboarding** — pluggable adapter scaffolded in `api/_payouts.ts` (default = simulator). `POST /api/connect/onboard` + `GET /api/connect/status/:payeeEmail` wired. Real Stripe Express impl pending `STRIPE_SECRET_KEY`.
 - ☒ **Royalty split sheets become real** — `applySplits()` in `api/_royalties.ts` distributes earnings per percentage; `POST /api/splits/:id/payout` triggers via the payout adapter (simulator pays after 2s, swaps to Stripe Transfers when key wired).
 - ☒ **Royalty ingestion pipeline** — `parseRoyaltyCsv()` handles common DSP CSV header variants (isrc / upc / streams / amount / etc); `POST /api/royalties/ingest` writes to in-memory ledger (Postgres `royalty_lines` table when `DATABASE_URL` set).
 - ☒ **Artist payout dashboard backing API** — `GET /api/earnings` returns real aggregates from the ledger (by platform / territory / period / release).
 - ☒ **Influencer / DJ payouts** — same payout adapter; per-verified-post payout flow ready.
 - ☒ **Tax document endpoint** — `GET /api/tax-docs/:payeeEmail` returns the Stripe-hosted document URL when Connect is live; placeholder otherwise.
-

Phase 4 — Creator Economy (1 week)

The Influencer / DJ portals exist as UI shells but don't connect to anything. Make them real.

- ☒ **Real influencer matching algorithm** — `api/_match.ts` replaces the `slice(0,2)` stub with a deterministic blended score over genre families, platform-per-genre weights, log-scaled reach (with micro-influencer 50K–250K bonus), and existing manual match score. Returns `ScoredInfluencer[]` with transparent reasons. `POST /api/influencers/match` wired. Embedding-based ranking is a drop-in upgrade later.
- [~] **Influencer post verification** — pluggable adapter scaffolded in `api/_post-verifier.ts` (default = manual review queue). TikTok + Meta adapters stubbed

pending `TIKTOK_ACCESS_TOKEN / META_ACCESS_TOKEN`. `POST`

`/api/influencers/:id/verify-post` + `GET /api/influencers/:id/posts`

wired; verified posts auto-trigger payout via the Phase 3 adapter.

- ☒ **DJ pack delivery** — `api/_signed-url.ts` issues HMAC-SHA256 signed URLs (24h TTL default). `POST /api/dj/packs/:packId/deliver` returns the signed URL; `GET /api/dj/packs/:packId/download` validates the signature in constant time. Swap to S3 / R2 presigned URLs when storage is wired — interface stays the same.
 - ☒ **DJ feedback loop** — `POST /api/dj/feedback` accepts 1-5 rating + comment + `willPlayInSet`. `GET /api/releases/:id/dj-feedback` aggregates into a `chartReadinessScore` artists can see.
 - [~] **Influencer / DJ Connect onboarding** — same payout adapter as Phase 3; per-role flag on `creator_accounts` table.
-

Phase 5 — Polish & Type Safety (3–5 days)

Now that the product does something real, make it not feel like a beta.

- ☐ **Loading skeletons** for every list/chart route — partially done (Messages page), full sweep pending
 - ☒ **Toast notifications** — `sonner` wired in `main.tsx` + used across the app
 - ☒ **Error boundaries** — `ErrorBoundary` component wraps the app
 - ☐ **Form validation with Zod + react-hook-form** — installed, partial use (NewRelease still uncontrolled)
 - ☒ **TanStack Query** — provider mounted in `main.tsx`, used in Messages
 - ☐ **Remove every any from the codebase** — partial; ~30 anys remain in store/app handlers
 - ☐ **Strict tsconfig** — pending
 - ☐ **Disabled-button states everywhere** — partial (AI Assistant, Messages); full sweep pending
 - ☐ **Empty states** — partial (Messages); full sweep pending
-

Phase 6 — Observability (1–2 days)

Catch problems before users tweet about them.

- ☐ **Sentry** — needs DSN; SDK install pending
 - ☒ **Vercel Analytics + Speed Insights** — wired in `main.tsx` (free, no key)
 - ☐ **PostHog** — needs project ID
 - ☒ **/api/health endpoint** — pings Postgres + selected LLM provider with 3s timeout, returns 503 if any fail
 - ☒ **Structured logging with Pino** — `api/_logger.ts` + `httpLog` middleware on every route
 - ☐ **Stripe webhook signature verification** — needs Phase 3 first
-

Phase 7 — Security Hardening

Some of these belong in Phase 0; the rest can come after a paying customer or two.

- ☐ **CSRF tokens** — pending Supabase Auth activation
 - ☒ **Rate limiting** — `api/_security.ts` token bucket on `/api/ai/chat` (30/5min), `/api/anr` (10/hr), `/api/dmca` (5/hr); Upstash swap-in ready
 - ☐ **Audio upload virus scan** — pending Cloudflare R2 / ClamAV setup
 - ☒ **DMCA takedown endpoint** — `POST /api/dmca` flags releases for review, audit-logged
 - ☒ **GDPR export + delete** — `GET /api/me/export` + `DELETE /api/me`
 - ☒ **Audit log** — middleware via `logAudit()` on takedown / metadata edit / export / delete / DMCA; readable at `GET /api/admin/audit`
 - ☒ **CSP + security headers** — `securityHeaders` middleware sets HSTS, X-Frame-Options, Referrer-Policy, Permissions-Policy, CSP
-

Out of scope (don't build until validated)

These were on the original 90-item plan. Skip them. Add only when something is actually slow or broken.

- ~~Docker / Kubernetes~~ — you're on Vercel, that's the deployment
- ~~Redis caching~~ — Postgres + HTTP cache headers are enough until you have 10k MAU

- ~~Service worker / offline support~~ — music *upload* tool, users are online
 - ~~Virtual scrolling~~ — only needed if a single artist has 10k+ releases (no one does)
 - ~~WebSockets~~ — Server-Sent Events (which you'll already have for AI streaming) cover the realtime needs
 - ~~A/B testing infrastructure~~ — premature; PostHog Experiments is one toggle when you need it
 - ~~Batch operations~~ — UX problem, not yet
 - ~~Advanced search~~ — Postgres full-text is enough until you have 50k releases
 - ~~Backup system~~ — Supabase / Neon ship daily PITR backups
 - ~~Custom CI/CD pipeline~~ — Vercel Git integration handles deploy + preview envs
-

Suggested timeline

If one engineer (or one Claude Code session per day) is working on this:

Week	Focus
1	Phase 0 — auth + DB + Zod
2	Phase 1 — DSP delivery + ISRC/UPC + audio validation
3	Phase 1 finish + Phase 2 AI integration start
4	Phase 2 finish — A&R critique, real strategy gen
5	Phase 3 — Stripe Connect + splits payouts
6	Phase 4 — influencer matching + post verification
7	Phase 5 — polish + types + skeletons
8	Phase 6 + 7 — Sentry, rate limiting, soft launch

That's a real product in 8 weeks. The original plan would still have you in Phase 1 at that point.

Definition of "shippable v1"

Before you charge anyone money, all of these must be true:

- ☐ An artist can sign up, upload audio + artwork, fill out metadata, and receive an email when the release is live on Spotify.
- ☐ That artist can see real analytics (plays, streams, top territories) on the release.
- ☐ That artist can chat with the AI assistant and get answers grounded in their own data.
- ☐ When the release earns money, the splits actually pay out via Stripe.
- ☐ An influencer can sign up, see matched campaigns, post the song, and get paid.
- ☐ If something breaks, Sentry catches it and you know before the artist tweets about it.

Everything else is nice-to-have.